PermitOS — Project Document

Band of Agents Hackathon | Track 3: Regulated & High-Stakes Workflows

Field	Value
Product	PermitOS
Tagline	The developer's multi-agent permitting team
Track	3 — Regulated & High-Stakes Workflows
Agents	5 specialized agents coordinated via Band
MVP Jurisdiction	Austin, Texas (city + county + state triggers)
Version	1.0
Status	Build specification

Table of Contents

1. Executive Summary
2. Problem Statement
3. Solution Overview
4. Product Scope
5. The Five Agents
6. System Architecture
7. Band Platform Integration
8. Data Models & Message Protocol
9. Knowledge Base
10. End-to-End Workflow
11. Technology Stack
12. Repository Structure

13. Logical Build Steps
14. Team Responsibilities
15. Demo Scenario
16. Audit, Compliance & Human Oversight
17. Business Model
18. Risks & Mitigations
19. Future Roadmap
20. Appendix

---

1. Executive Summary

PermitOS is a multi-agent permitting command center for real estate developers, general contractors, and architecture firms. Instead of hiring permit expediters and manually navigating hundreds of overlapping regulations, a developer uploads project details and plans. Five specialized AI agents—coordinated through the **Band SDK**—analyze jurisdiction rules, building codes, environmental triggers, and utility requirements in parallel. They merge findings, resolve conflicts, assemble a complete permit package with fee estimates and filing sequence, and route the result to a human for final approval with a full audit trail.

PermitOS is built for **Track 3** of the Band of Agents Hackathon: regulated, high-stakes workflows where traceability, escalation, and careful decision-making are mandatory. Band is not a notification layer—it is the active coordination bus where agents discover each other, hand off tasks, share structured context, and involve humans at the right gates.

MVP focus: One jurisdiction executed deeply (Austin, TX), with architecture designed to scale to additional cities post-hackathon.

---

2. Problem Statement

The permitting bottleneck

United States construction is a **$2.1 trillion** industry. Before ground breaks, every project must pass through a permitting gauntlet:

- **2–12 months** average processing time per jurisdiction
- **200+ laws and ordinances** governing a single land development project
- **23–36 separate regulatory items** for a typical building permit (more for factories, commercial, mixed-use)
- **$50,000–$150,000** spent per project on permit expediters and consultants
- **$10,000–$30,000 per month** in carrying costs for every month of delay

Who feels the pain

- Real estate developers waiting on entitlements
- General contractors blocked from scheduling trades
- Architecture firms producing plans that fail review for fixable code issues
- Municipalities overwhelmed (cities are now buying AI—Clariti, South Korea national system—but the **developer side** has no equivalent multi-agent product)

Why now

Cities are deploying AI on the **reviewer side** (Honolulu: 70% faster plan review with CivCheck). Developers still operate on email, spreadsheets, and expensive consultants. The gap between "city has AI" and "developer has AI" is widening. PermitOS fills the developer-side gap with a governed, auditable multi-agent system.

---

3. Solution Overview

PermitOS gives every development project a **virtual permitting department**:

Traditional	PermitOS
Manual code research across jurisdictions	Agents load curated rule packs + RAG
Sequential reviews (zoning, then fire, then env)	Agents 2–4 run in parallel via Band
Conflicts discovered at submission	Conductor detects conflicts before filing
Incomplete applications returned by city	Packager ensures checklist completeness
No audit trail of who checked what	Every agent decision logged with citations
Human approves implicitly at filing	Explicit human gate before "ready to file"

Core value proposition

Faster: Parallel analysis cuts weeks of sequential consultant work to minutes.

Cheaper: Replaces a portion of permit expediter fees.

Safer: Citations, audit trail, and human approval satisfy regulated-workflow requirements.

Scalable: Add jurisdictions by adding knowledge packs, not rebuilding agents.

---

4. Product Scope

In scope (MVP)

- Web dashboard: project intake, live agent activity, compliance report, permit package
- Five Band-connected agents with distinct roles
- Austin, TX knowledge pack (zoning, building, environmental, utilities, permit catalog, fees)
- Structured JSON outputs from every agent
- Conflict detection and resolution suggestions
- Human approval gate with audit hash
- Optional PDF plan text extraction
- Demo video and submission-ready repository

Out of scope (MVP)

- Live submission to government permit portals
- All 50 US states
- Professional engineer / architect stamp replacement
- Full CAD/BIM parsing
- Legal advice (system is **pre-screen only**)

Disclaimer (shown in product)

> PermitOS provides pre-screening assistance only. It does not constitute legal, engineering, or architectural advice. All filings require review by licensed professionals and approval by applicable authorities.

---

5. The Five Agents

Five agents replace an original eight-agent design by merging related domains. Each agent is a separate Band participant with its own system prompt, tools, and (preferably) a different agent framework or LLM provider.

Agent 1 — Conductor

Role: Orchestrator, conflict resolver, human gate

Attribute	Detail
Framework	LangGraph or PydanticAI
Model	Strong reasoning (Claude / GPT-4 class)
Triggers workflow	Yes — receives intake from API/UI

Responsibilities:

- Receive project intake; create Band chatroom per permit case
- Parse intake into structured `ProjectBrief` JSON
- Invite specialist agents to the room
- Dispatch scoped tasks via `@mention` (not raw dumps of entire files)
- Wait for `type: complete` messages from specialists
- Merge reports; run conflict detection rules
- Compute readiness: `READY` | `NEEDS_CHANGES` | `BLOCKED`
- Escalate to human for final approval or critical conflicts
- Publish executive summary to dashboard

Outputs: `PermitCaseSummary` — case_id, status, readiness_score, conflicts[], human_actions_required[], audit_events[]

Band tools: `thenvoi_create_chatroom`, `thenvoi_add_participant`, `thenvoi_send_message`, `thenvoi_send_event`, `thenvoi_get_participants`, `thenvoi_lookup_peers`

Agent 2 — Jurisdiction & Zoning

Role: Which rules apply; land use and zoning compliance

Attribute	Detail
Framework	CrewAI or Anthropic adapter

Model	Different vendor from Conductor (cross-model bonus)

Responsibilities:

- Resolve address → city, county, state, special districts
- Load jurisdiction rule pack from knowledge base
- Check zoning district, permitted use, setbacks, FAR, height, lot coverage, parking, density
- Flag by-right vs variance/CUP requirement
- Cite specific code sections on every finding

Inputs: address, project type, lot size, unit count, stories

Outputs: `JurisdictionReport` — jurisdictions[], zoning{}, checks[], blockers[], data_gaps[]

Tools: `lookup_jurisdiction`, `get_zoning_rules`, `calculate_setbacks`

Agent 3 — Building & Safety

Role: Building code, fire, egress, accessibility pre-screen

Attribute	Detail
Framework	LangGraph or OpenAI adapter
Model	Different from Agent 2 if possible

Responsibilities:

- Pre-screen IBC/IRC (residential) + local amendments
- Fire: egress, sprinkler triggers, fire lane access
- Accessibility: ADA/FHA accessible unit percentage, routes
- Structural red flags from brief (not PE stamp)
- Every finding: pass | fail | warn + citation

Inputs: ProjectBrief, unit mix, stories, sq ft, parking

Outputs: `BuildingSafetyReport` — checks[], blockers[], recommendations[]

Tools: `get_building_code_rules`, `check_egress_requirements`, `check_accessibility_requirements`

Agent 4 — Site, Environmental & Utilities

Role: Environmental triggers + site infrastructure

Attribute	Detail
Framework	PydanticAI or Gemini adapter
Model	Third provider if budget allows

Responsibilities:

- Environmental screening: wetlands, FEMA flood zone, endangered species triggers, stormwater, noise (CEQA/NEPA screen level)
- Utilities: water/sewer capacity, electrical service, gas, storm drain, traffic impact screen
- List additional permits beyond building permit

Inputs: address, site acreage, unit count, impervious surface estimate

Outputs: `SiteEnvironmentalReport` — environmental_checks[], utility_checks[], additional_permits[]

Tools: `lookup_flood_zone`, `get_environmental_triggers`, `get_utility_requirements`

Agent 5 — Permit Packager & Tracker

Role: Assemble submission package; track status; draft RFI responses

Attribute	Detail
Framework	Claude SDK or Codex adapter
Model	Strong document generation

Responsibilities:

- Consume merged reports from Conductor
- Generate permit checklist: forms, documents, agencies, fees, timelines
- Calculate fee estimates from fee schedule
- Produce filing sequence with dependencies

- Simulate post-submission status (Submitted → In Review → RFI → Approved)
- Draft RFI responses when city requests clarification

Inputs: PermitCaseSummary + all specialist reports

Outputs: `PermitPackage` — permits_required[], documents_required[], total_fees_estimate, estimated_timeline_days, filing_sequence[], rfi_draft (optional)

Tools: `get_permit_templates`, `get_fee_schedule`, `generate_checklist`, `draft_rfi_response`

Agent interaction summary

```
■■■■■■■■■■■■■■■■■■■■■ ■ CONDUCTOR ■ ■ (orchestrator) ■ ■■■■■■■■■■■■■■■■■■■■■ ■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■ parallel ■ ■ ▼ ▼ ▼
■■■■■■■■■■■■■■■■■■ ■■■■■■■■■■■■■■■■■■ ■■■■■■■■■■■■■■■■■■ ■ Jurisdiction ■ ■
Building ■ ■ Site & Env ■ ■ & Zoning ■ ■ & Safety ■ ■ & Utilities ■ ■■■■■■■■■■■■■■■■■■
■■■■■■■■■■■■■■■■■■ ■■■■■■■■■■■■■■■■■■ ■ ■ ■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■ merge ▼ ■■■■■■■■■■■■■■■■■■■■■ ■
PACKAGER ■ ■ & Tracker ■ ■■■■■■■■■■■■■■■■■■■■■ ■ ▼ ■■■■■■■■■■■■■■■■■■■■■ ■ HUMAN
APPROVAL ■ ■■■■■■■■■■■■■■■■■■■■■
```

---

6. System Architecture

High-level diagram

```
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■
DEVELOPER UI ■ ■ Upload project ■ Live agent feed ■ Reports ■ Approve ■ Export ■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■
REST / WebSocket
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■▼■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■
API LAYER (FastAPI) ■ ■ POST /cases ■ GET /cases/{id} ■ POST /approve ■ Webhooks ■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■ ■ ■ ▼ ▼ ▼ ■■■■■■■■■■■■■■■■■■
■■■■■■■■■■■■■■■■■■ ■■■■■■■■■■■■■■■■■■ ■ Database ■ ■ File Storage ■ ■
Knowledge/RAG ■ ■ cases, audit ■ ■ PDF uploads ■ ■ Austin rules ■ ■■■■■■■■■■■■■■■■■■
■■■■■■■■■■■■■■■■■■ ■■■■■■■■■■■■■■■■■■ ■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■▼■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ ■
BAND CHATROOM (one per permit case) ■ ■ ■ ■ Conductor ■■■■ Jurisdiction ■■■■ Building
■■■■ Site/Env ■ ■ ■ ■ ■ ■■■■■■■■■■■■■■■■■■■■ Packager ■■■■ Human (in-the-loop) ■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
```

Component responsibilities

Component	Responsibility

UI	Intake forms, real-time status, compliance dashboard, approval button, PDF/HTML export
API	Case lifecycle, persistence, bridge between UI and Conductor trigger
Database	Cases, agent reports, audit log, human decisions
Knowledge base	Curated Austin rules, fees, permit catalog for RAG + deterministic lookups
Band	Agent-to-agent messaging, room lifecycle, events, human participant
Agents	Domain analysis, structured output, tool execution

---

7. Band Platform Integration

Band must be the **active coordination layer**, not a thin wrapper around a single LLM. Judges require deep SDK integration.

Requirements checklist

#	Requirement	Implementation
1	3+ specialized agents	5 agents
2	Agents actively communicate	`@mention` handoffs in chatroom
3	Task handoffs	Conductor → specialists → Packager
4	Shared context	Structured JSON in Band messages
5	Agent discovery	`thenvoi_lookup_peers`, `thenvoi_get_participants`

6	State changes	`thenvoi_send_event` for progress
7	Room per workflow	`thenvoi_create_chatroom` per case
8	Cross-framework	LangGraph + PydanticAI + CrewAI (mix)
9	Human-in-the-loop	Human added via `thenvoi_add_participant`

Band tools per agent

Tool	Used by	Purpose
`thenvoi_create_chatroom`	Conductor	New permit case room
`thenvoi_add_participant`	Conductor	Add agents + human
`thenvoi_send_message`	All	Task dispatch, reports, `@mentions`
`thenvoi_send_event`	All	Progress, errors, status
`thenvoi_get_participants`	Conductor	Verify room membership
`thenvoi_lookup_peers`	Conductor	Discover registered agents

Room lifecycle

1. **Create** — Conductor creates `permit-case-{uuid}` when intake received
2. **Populate** — Add Agents 2–5 and optionally human observer
3. **Dispatch** — Conductor posts scoped tasks with `@mentions`
4. **Parallel work** — Agents 2–4 analyze; post events + complete messages
5. **Merge** — Conductor synthesizes; posts summary
6. **Package** — Conductor `@mentions` Packager; Packager returns package
7. **Approve** — Human approves in room or via UI (posted to Band)

8. **Archive** — Room persisted; audit log finalized

---

8. Data Models & Message Protocol

ProjectBrief (intake)

```
{ "case_id": "uuid", "project_name": "Riverside Residences", "address": "1200 Riverside Dr, Austin, TX 78704", "project_type": "multifamily_residential", "units": 50, "stories": 4, "gross_sqft": 52000, "lot_sqft": 85000, "parking_spaces": 75, "plan_pdf_url": "optional", "notes": "optional" }
```

Band message envelope (every agent message)

```
{ "type": "finding | complete | question | conflict", "case_id": "uuid", "agent": "conductor | jurisdiction | building | site | packager", "timestamp": "ISO-8601", "payload": {}, "citations": [ { "source": "Austin LDC 25-2-491", "url": "https://..." } ] }
```

Specialist report pattern

```
{ "agent": "jurisdiction", "case_id": "uuid", "summary": "Setback violation on Block B", "readiness_impact": "needs_changes", "checks": [ { "rule": "Side setback minimum 10ft", "status": "fail", "citation": "Austin LDC 25-2-491", "detail": "Block B east wall at 8ft" } ], "blockers": ["Setback non-compliance Block B"], "data_gaps": [] }
```

PermitPackage (final output)

```
{ "case_id": "uuid", "permits_required": [ { "agency": "City of Austin DSD", "permit_name": "Building Permit", "form_id": "BP-2026", "fee_usd": 12400, "timeline_days": 30, "dependencies": ["zoning_verification"] } ], "documents_required": [ { "name": "Site plan", "source_agent": "jurisdiction", "status": "required" } ], "total_fees_estimate_usd": 47200, "estimated_timeline_days": 45, "filing_sequence": ["Step 1: Zoning verification", "Step 2: ..."], "audit_hash": "sha256:..." }
```

---

9. Knowledge Base

MVP: Austin, Texas

Curate quality over quantity. **15–30 accurate rules beat 500 scraped fragments.**

```
knowledge/austin/ ■■■ zoning_rules.json # Districts, setbacks, FAR, height, parking ■■■ building_code_snippets/ # 20–30 excerpts with citations ■■■ environmental_triggers.json #
```

```
Flood, wetland, stormwater thresholds ■■■ utility_requirements.json # Water, sewer,
electric triggers ■■■ permit_catalog.json # Permit types, agencies, forms, dependencies
■■■ fee_schedule.json # 2026 fee estimates
```

RAG strategy

- Embed code snippets + permit catalog into vector store (Chroma or similar)
- Agents query RAG for contextual rules; deterministic tools for calculations (setbacks, fees)
- Every agent response must include `citations[]` — empty citations = invalid output

Geocoding

- Address → lat/lng → jurisdiction lookup
- MVP fallback: if address contains "Austin, TX", load Austin pack; else run generic IBC checks with warning "demo jurisdiction only"

---

10. End-to-End Workflow

Step 1 — Project intake

Developer submits project via UI. API creates `case_id`, stores `ProjectBrief`, triggers Conductor.

Step 2 — Room creation & dispatch

Conductor creates Band chatroom, adds specialist agents, posts scoped tasks with `@mentions`.

Step 3 — Parallel analysis

Agents 2 (Jurisdiction), 3 (Building), and 4 (Site/Env) work in parallel. Each posts progress events, then `type: complete` with structured report.

Step 4 — Merge & conflict resolution

Conductor ingests three reports. Conflict rules examples:

Condition	Result
Zoning fail (blocker)	`BLOCKED`
Building fail, no blockers elsewhere	`NEEDS_CHANGES`

Site warn only	`NEEDS_CHANGES` or `READY` with notes
Contradictory citations	`conflict` object + suggested resolution

Step 5 — Permit packaging

Conductor `@mentions` Packager with merged context. Packager returns `PermitPackage` with checklist, fees, filing sequence.

Step 6 — Human approval gate

Conductor posts executive summary. Human reviews in UI; clicks Approve (posted to Band). System generates audit hash. Status → `APPROVED_FOR_FILING`.

Step 7 — Post-submission (optional demo)

Simulate city RFI. Packager drafts response. Tracker updates timeline. Audit log records RFI handling.

---

11. Technology Stack

Layer	Technology	Rationale
Language	Python 3.11+	Band SDK native
Band SDK	thenvoi / Band Python SDK	Hackathon requirement
Agent frameworks	LangGraph, PydanticAI, CrewAI	Cross-framework story
LLM APIs	Anthropic, OpenAI, Google	Cross-model agents
Backend	FastAPI	Async, WebSocket, OpenAPI
Frontend	Next.js or React (Vite)	Dashboard, upload, live feed
Database	SQLite (dev) / PostgreSQL (prod)	Cases, audit
Vector DB	Chroma	Austin code RAG

PDF extract	PyMuPDF	Plan text extraction
Geocoding	Google Maps API or Nominatim	Address resolution
Deploy	Docker Compose	Agents + API + UI
CI	GitHub Actions	Lint, test

---

12. Repository Structure

```
permitos/ ■■■ agents/ ■ ■■■ conductor/ ■ ■ ■■■ agent.py ■ ■ ■■■ prompts/ ■ ■ ■■■
agent_config.yaml ■ ■■■ jurisdiction/ ■ ■■■ building/ ■ ■■■ site_environmental/ ■ ■■■
packager/ ■■■ shared/ ■ ■■■ schemas/ # Pydantic models ■ ■■■ band_client/ # Band
connection helpers ■ ■■■ tools/ # Shared tool implementations ■■■ knowledge/ ■ ■■■
austin/ # Rule packs ■■■ api/ ■ ■■■ main.py ■ ■■■ routes/ ■ ■■■ services/ ■■■ web/ #
Frontend app ■■■ tests/ ■■■ docker-compose.yml ■■■ .env.example ■■■ README.md
```

---

13. Logical Build Steps

Build in dependency order. Complete each phase before moving to the next. Phases can overlap across team members where noted.

Phase 1 — Foundation

Goal: Repository, schemas, and Band connectivity proven.

Step	Task	Done when
1.1	Create repository structure	Folders, README, .env.example exist
1.2	Define Pydantic schemas	ProjectBrief, all reports, PermitPackage, Band envelope
1.3	Register 5 agents on Band platform	5 agent IDs + API keys in config

1.4	Implement shared Band client	Connect, join room, send/receive message
1.5	Prove one round-trip	Conductor sends message; one agent replies in Band room
1.6	Docker Compose skeleton	API + one agent container starts

Phase 2 — Knowledge layer

Goal: Austin rule data agents can query.

Step	Task	Done when
2.1	Curate Austin zoning_rules.json	Districts, setbacks, FAR for demo scenario
2.2	Curate building_code_snippets	20+ excerpts with real citations
2.3	Curate environmental_triggers.json	Flood, wetland, stormwater rules
2.4	Curate utility_requirements.json	Capacity thresholds
2.5	Curate permit_catalog.json + fee_schedule.json	Permits, forms, fees for demo
2.6	Build RAG index	Agents retrieve relevant snippets
2.7	Implement deterministic tools	lookup_jurisdiction, calculate_setbacks, lookup_flood_zone

Phase 3 — Specialist agents (2, 3, 4)

Goal: Three agents produce real structured reports.

Step	Task	Done when

3.1	Jurisdiction agent: prompt + tools + Band adapter	Returns valid JurisdictionReport JSON
3.2	Building agent: prompt + tools + Band adapter	Returns valid BuildingSafetyReport JSON
3.3	Site/Env agent: prompt + tools + Band adapter	Returns valid SiteEnvironmentalReport JSON
3.4	Wire agents to respond to Conductor `@mentions`	Dispatch triggers analysis
3.5	Validate citations on all outputs	No empty citation arrays

Phase 4 — Conductor agent

Goal: Orchestration, merge, conflict detection.

Step	Task	Done when
4.1	Intake handler: ProjectBrief → Band room	Room created per case
4.2	Parallel dispatch to agents 2–4	Three `@mentions` with scoped briefs
4.3	Wait for `type: complete` from all three	Timeout + retry handling
4.4	Merge logic + conflict rules	PermitCaseSummary generated
4.5	Readiness score computation	READY / NEEDS_CHANGES / BLOCKED
4.6	Escalation to human	Summary posted; human `@mentioned`

Phase 5 — Packager agent

Goal: Permit package from merged reports.

Step	Task	Done when
5.1	Packager responds to Conductor handoff	Triggered after merge
5.2	Checklist generation from permit_catalog	All required permits listed
5.3	Fee calculation from fee_schedule	Total fees in output
5.4	Filing sequence with dependencies	Ordered steps
5.5	RFI draft capability	Demo act 2 works

Phase 6 — API & persistence

Goal: Backend stores cases and triggers workflow.

Step	Task	Done when
6.1	POST /cases — create case, trigger Conductor	End-to-end from API
6.2	GET /cases/{id} — status + reports	UI can poll
6.3	POST /cases/{id}/approve — human gate	Approval stored + posted to Band
6.4	Audit log persistence	Every message + decision stored
6.5	Audit hash generation	SHA-256 of final package

Phase 7 — Frontend

Goal: Demo-ready dashboard.

Step	Task	Done when
7.1	Intake form	Address, units, stories, PDF upload
7.2	Live agent activity feed	Shows Band events / status
7.3	Compliance report view	Pass/fail/warn per rule with citations
7.4	Conflict panel	Conductor conflicts visible
7.5	Permit package view	Checklist, fees, timeline
7.6	Approve button	Human gate in UI
7.7	Export report	PDF or HTML download

Phase 8 — Integration & hardening

Goal: Reliable end-to-end demo.

Step	Task	Done when
8.1	Full pipeline test with demo scenario	Riverside Residences runs clean
8.2	Error handling: agent timeout, malformed JSON	Graceful degradation
8.3	Band seam fixes: auth refresh, message parse	No silent failures
8.4	Performance: parallel agents, RAG cache	Demo completes in < 3 min
8.5	Legal disclaimer in UI	Visible on every report

Phase 9 — Demo & submission

Goal: Hackathon deliverables complete.

Step	Task	Done when
9.1	Architecture diagram	In README + slide
9.2	Demo video (3–5 min)	Recorded and uploaded
9.3	README: setup for judges	`docker compose up` works
9.4	lablab.ai submission	Form complete
9.5	GitHub public repo	Clean, licensed

---

14. Team Responsibilities

Role	Owns	Delivers
Tech Lead / Band	Conductor, Band SDK, schemas, Docker	Agents coordinate in Band
Agent Dev A	Jurisdiction + Building agents	Reports with citations
Agent Dev B	Site/Env + Packager agents	Reports + permit package
Knowledge Lead	Austin rule packs, demo scenario	Accurate citations
Backend Dev	FastAPI, database, audit	API + persistence
Frontend Dev	Dashboard, intake, approval UI	Demo interface
Demo Lead	Video, diagram, submission copy	Hackathon deliverables

Roles can merge on smaller teams. Minimum viable team: 3 people (Band+agents, knowledge+backend, frontend+demo).

---

15. Demo Scenario

Project: Riverside Residences

Field	Value
Address	1200 Riverside Dr, Austin, TX 78704
Type	50-unit multifamily, 4 stories
Intentional flaw	Block B violates side setback by 2 feet

Expected agent outputs

Agent	Key finding
Jurisdiction	MF-3 zoning OK; setback FAIL Block B (cite Austin LDC)
Building	Egress PASS; sprinklers REQUIRED
Site/Env	Flood zone X; no wetland; sewer capacity WARN
Conductor	NEEDS_CHANGES; suggest reduce footprint or variance
Packager	23 permits, ~$47,200 fees, 45-day estimate

Demo flow (3–5 minutes)

1. State the problem (permitting delay, cost)
2. Upload project in UI; Band room created
3. Show parallel agents analyzing (live feed)
4. Show compliance dashboard with pass/fail/citations
5. Show Conductor conflict + suggested fix
6. Show permit package (checklist + fees)
7. Human approves; audit hash displayed

8. (Optional) Simulate city RFI; Packager drafts response
9. Architecture: 5 agents, Band bus, Track 3 compliance

---

16. Audit, Compliance & Human Oversight

Track 3 alignment

Requirement	PermitOS implementation
Traceability	Every check logged with agent, timestamp, citation
Escalation	Conductor escalates blockers and final approval to human
Careful decisions	No auto-file; human APPROVE required
Audit trail	Band message IDs + local DB + SHA-256 package hash
Regulated workflow	Pre-screen disclaimer; citations mandatory

Audit log fields

- case_id, timestamp, agent_id, event_type, message_id (Band), payload_hash, human_id (if applicable)

Human gate rules

- No status `APPROVED_FOR_FILING` without explicit human action
- UI Approve button posts to Band room (not side-channel only)
- REQUEST_CHANGES loops back to Conductor with human notes

---

17. Business Model

Target customers

- Real estate developers (5–200 units/project)

- General contractors
- Architecture firms with permitting overhead
- Permit expediters (white-label)

Pricing (post-hackathon)

Model	Price
Per project	$2,000–$15,000 depending on complexity
Subscription	$499–$2,999/month per organization
Enterprise	Custom + API access

ROI for customer

- Save 2–8 weeks permit prep → $20,000–$240,000 carrying cost avoided
- Reduce expediter fees $50,000+ per project
- Fewer incomplete submissions → fewer city rejections

---

18. Risks & Mitigations

Risk	Impact	Mitigation
Band SDK issues	Agents don't coordinate	Prove round-trip in Phase 1; record demo video early
Agents return prose not JSON	Merge fails	Strict schemas; parse/retry; validator layer
Inaccurate code citations	Credibility loss	Curate knowledge pack; human review Austin data
Slow live demo	Bad judging impression	Cache RAG; pre-warm agents; shorten brief

Legal liability	User trust	Prominent disclaimer; pre-screen only
Scope creep	Miss submission	Austin only; 5 agents max; defer portal integration

---

19. Future Roadmap

Phase	Scope
Phase 1	Texas triangle: Austin, Dallas, Houston
Phase 2	Accela / OpenCounter portal integrations
Phase 3	White-label for architects and GCs
Phase 4	Multiplayer Band rooms: city reviewer agent + developer agent negotiate RFIs
Phase 5	National expansion via jurisdiction knowledge marketplace

---

20. Appendix

A. Agent quick reference

#	Agent	Input	Output	Waits for
1	Conductor	ProjectBrief	PermitCaseSummary	Specialists → Packager
2	Jurisdiction & Zoning	Scoped brief	JurisdictionReport	Conductor dispatch
3	Building & Safety	Scoped brief	BuildingSafetyReport	Conductor dispatch

4	Site, Env & Utilities	Scoped brief	SiteEnvironmentalReport	Conductor dispatch
5	Permit Packager & Tracker	Merged reports	PermitPackage	Conductor merge

B. Conflict detection rules (Conductor)

```
IF any agent.blockers.length > 0 AND blocker.category == "zoning": readiness = BLOCKED ELIF any agent.checks.status == "fail": readiness = NEEDS_CHANGES ELIF any agent.checks.status == "warn" COUNT > 2: readiness = NEEDS_CHANGES ELSE: readiness = READY
```

C. Environment variables

```
BAND_API_KEY= BAND_AGENT_ID_CONDUCTOR= BAND_AGENT_ID_JURISDICTION= BAND_AGENT_ID_BUILDING= BAND_AGENT_ID_SITE= BAND_AGENT_ID_PACKAGER= ANTHROPIC_API_KEY= OPENAI_API_KEY= GOOGLE_API_KEY= # optional DATABASE_URL= GEOCODING_API_KEY= # optional
```

D. Hackathon submission checklist

- [ ] GitHub repository (public)
- [ ] README with setup instructions
- [ ] Architecture diagram
- [ ] Demo video (3–5 min)
- [ ] 5 agents visible in Band
- [ ] Track 3: audit trail + human gate demonstrated
- [ ] lablab.ai submission form completed

Document end.

PermitOS — Band of Agents Hackathon — Track 3